

Instructions

- Homework 3 was due Monday, August 10th at 18:00 Chicago Time.
 - All numbers and figures come from `homework_3_solution.ipynb`, run from top to bottom.
-

Problem 1: Ridge Regression and Stability

Generate a dataset with $n = 100$ observations based on the true model:

$$y = 1 + x_1 + x_2 + \epsilon$$

where $\epsilon \sim \mathcal{N}(0, 1)$.

Additionally, make $x_1, x_2 \sim \mathcal{N}(0, 1)$ highly correlated, with $\rho = 0.99999$. You can do this via the following code snippet:

```
1 import numpy as np
2 np.random.seed(1)
3
4 mean = [0, 0]
5 cov = [[1, 0.99999], [0.99999, 1]]
6 x1, x2 = np.random.multivariate_normal(mean, cov, 100).T
7 epsilon = np.random.normal(0, 1, 100)
```

Problem 1.1: OLS Inference

Fit an OLS model to your generated data using `statsmodels`.

- Report the p-values and confidence intervals for $\hat{\beta}_1$ and $\hat{\beta}_2$.
- Discuss the results. Are the coefficients close to the true values (1 and 1)? Are they statistically significant?

Answer:

$$R^2 = 0.733.$$

	coef	std err	t	P > t	[0.025	0.975]
const	0.9940	0.105	9.483	0.000	0.786	1.202
X1	24.1321	24.274	0.994	0.323	-24.046	72.310
X2	-22.1958	24.257	-0.915	0.362	-70.340	25.948

Neither slope is close to 1, and neither is significant: $p = 0.323$ and $p = 0.362$, with intervals almost 100 units wide. Yet the intercept is estimated precisely and $R^2 = 0.733$, so the model as a whole fits.

That is multicollinearity. With $\rho = 0.99999$ the data cannot separate β_1 from β_2 , only their sum, and $\hat{\beta}_1 + \hat{\beta}_2 = 1.94$ against a true sum of 2.

Problem 1.2: Stability

Now, re-generate your data using `np.random.seed(42)` and fit the OLS model again.

- Report the new estimates for $\hat{\beta}_1$ and $\hat{\beta}_2$.
- Compare these estimates to the previous ones. What do you notice?

Answer:

	coef	std err	t	P> t	[0.025	0.975]
const	1.0928	0.108	10.115	0.000	0.878	1.307
X1	39.3968	24.079	1.636	0.105	-8.393	87.187
X2	-37.5875	24.083	-1.561	0.122	-85.386	10.211

Changing the seed moves $\hat{\beta}_1$ from 24.13 to 39.40 and $\hat{\beta}_2$ from -22.20 to -37.59 . The individual coefficients are not reproducible at all.

The sums are close: 1.94 before, 1.81 now. Only the sum is identified.

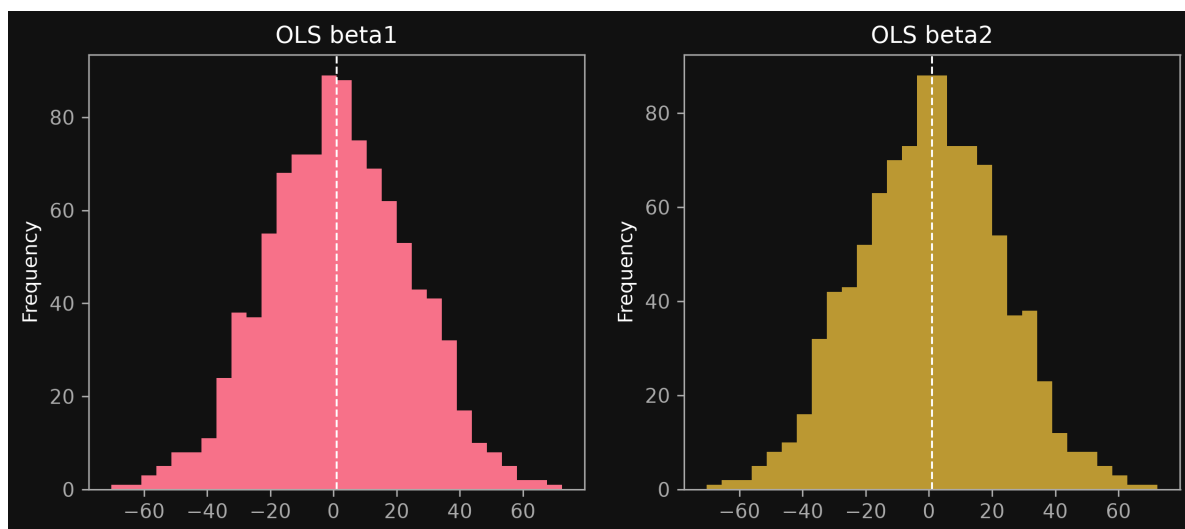
Problem 1.3 Variance via Simulation

To better understand this phenomenon, run a simulation (note, you will need to unset the random seed for this part):

- Run a simulation loop 1000 times. In each iteration, generate a new dataset (x_1, x_2, ϵ) and fit an OLS model.
- Store the estimates for $\hat{\beta}_1$ and $\hat{\beta}_2$.

Plot 2 histograms showing the distribution of the estimates for $\hat{\beta}_1$ and $\hat{\beta}_2$. Additionally, report the volatility (standard deviation) of your $\hat{\beta}_1$ and $\hat{\beta}_2$ estimates across the simulations.

Answer:



Standard deviation across the 1000 fits: 22.4496 for $\hat{\beta}_1$ and 22.4495 for $\hat{\beta}_2$, against a true value of 1 for each. The two histograms are mirror images because the errors cancel: $\hat{\beta}_1$ and $\hat{\beta}_2$ are almost perfectly negatively correlated, so their sum stays near 2 while each one wanders with a standard deviation of 22.

Problem 1.3: Ridge to the Rescue

Now, fit a Ridge Regression model using `statsmodels` (you can use `fit_regularized` with `L1_wt=0` for Ridge).

- Use three different values for alpha (lambda): 0.01, 0.1, and 1.
- For each alpha, report the coefficients $\hat{\beta}_1$ and $\hat{\beta}_2$.
- Discuss how the coefficients change with different alpha values.

Answer:

Fitted on the seed-1 dataset from 1.1, so these are directly comparable to the 24.13 and -22.20 reported there.

α	const	X1	X2
0.01	0.9784	0.9720	0.9324
0.10	0.8713	0.8883	0.8850
1.00	0.3997	0.5358	0.5358

A penalty of 0.01 is enough. The coefficients go from 24.13 and -22.20 to 0.9720 and 0.9324, both within 7% of the truth.

The penalty breaks the tie between the two collinear columns. OLS is free to put a huge positive weight on one and cancel it with a huge negative weight on the other, because the residual sum of squares barely notices; ridge charges $\alpha\|\beta\|^2$ for that, so the cheapest way to produce the same fitted values is to split the effect evenly. Numerically, this is the same statement as $X^\top X + \alpha I$ being far better conditioned than $X^\top X$.

Past that point α costs accuracy: by $\alpha = 1$ both coefficients have been shrunk to 0.5358 and the intercept to 0.3997.

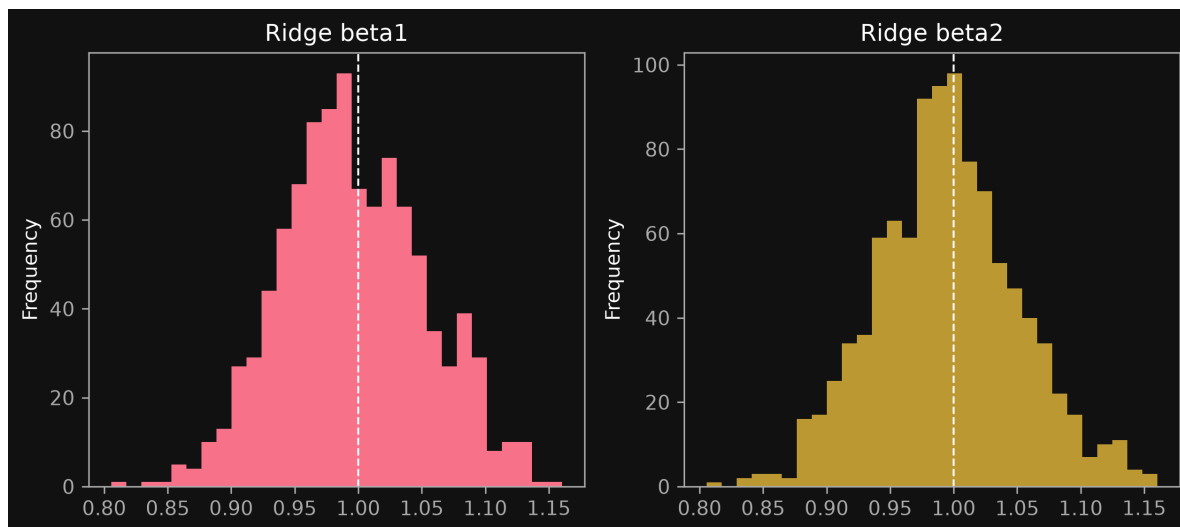
Problem 1.4: Ridge Simulation

Next, you should repeat the simulation from subproblem 1.3, but this time use a Ridge model with a fixed alpha of 0.01.

- Run the simulation loop 1000 times, generating new datasets and fitting the Ridge model each time.
- Store the estimates for $\hat{\beta}_1$ and $\hat{\beta}_2$.

Plot 2 histograms showing the distribution of the Ridge estimates for $\hat{\beta}_1$ and $\hat{\beta}_2$. Additionally, report the volatility (standard deviation) of your Ridge $\hat{\beta}_1$ and $\hat{\beta}_2$ estimates across the simulations. Compare these volatilities to those obtained from the OLS simulation in subproblem 1.3.

Answer:



	OLS	Ridge ($\alpha = 0.01$)
$\text{sd}(\hat{\beta}_1)$	22.4496	0.0563
$\text{sd}(\hat{\beta}_2)$	22.4495	0.0562

Both simulations use the same seed, so they run on identical datasets. A penalty of 0.01 cuts the volatility by a factor of 400: the estimates now sit on the true value of 1, against a cloud with a standard deviation of 22.

Ridge is biased and OLS is not. On this data that trade is worth taking many times over.

Problem 2: Lasso Regression and Sparsity

This problem explores Lasso regression for a sparse model.

Generate a dataset with $n = 100$ observations and $p = 50$ features. The true model depends on only the first 3 features, while the remaining 47 features are irrelevant.

$$y = 5x_1 - 2x_2 + 3x_3 + \epsilon$$

where $\epsilon \sim \mathcal{N}(0, 1)$ and all $x_j \sim \mathcal{N}(0, 1)$.

Use the following code snippet to generate your data:

```
1 import numpy as np
2 np.random.seed(42)
3
4 n_samples = 100
5 n_features = 50
6
7 X = np.random.normal(0, 1, (n_samples, n_features))
8 true_beta = np.zeros(n_features)
9 true_beta[:3] = [5, -2, 3]
10
11 y = np.dot(X, true_beta) + np.random.normal(0, 1, n_samples)
```

Problem 2.1: Naive OLS

Fit an OLS model using all 50 features.

- Look at the coefficients for the 47 “noise” features (x_4 through x_{50}). Are they exactly zero (you don’t need to report them all)?
- How many of these noise features have p-values < 0.05 (i.e., appear statistically significant purely by chance)?

Answer:

None of the 47 are exactly zero – the largest is 0.4715 in absolute value. OLS minimizes the residual sum of squares, and setting a coefficient to exactly zero is a measure-zero event.

4 of the 47 have $p < 0.05$. At a 5% level on pure noise you expect $47 \times 0.05 = 2.35$, so 4 is unremarkable – which is the point. Testing 47 irrelevant features one at a time will hand you a few “significant” ones every time.

Problem 2.2: Lasso for Feature Selection

Now, fit a Lasso model using `statsmodels`. You can do this using `fit_regularized` with `L1_wt=1` (which specifies pure Lasso). Use an alpha of 0.1, 1.0, and 100.

- Report the estimated coefficients.
- How many coefficients are estimated to be exactly zero?
- Did the Lasso model correctly identify the 3 relevant features (x_1, x_2, x_3) while suppressing the noise?
- Which alpha value would you recommend?

Answer:

α	exactly zero	kept
0.1	30 of 50	X_1, X_2, X_3 and 17 noise features
1.0	47 of 50	X_1, X_2, X_3
100	50 of 50	nothing

$\alpha = 1$ recovers the support exactly: the three real features and nothing else. The fitted model is

$$\hat{y} = 3.7783x_1 - 0.9403x_2 + 2.0392x_3.$$

So $\alpha = 1$ is the recommendation, with the caveat that the surviving coefficients are heavily shrunk against the true 5, -2 and 3. If you want the values as well as the support, use Lasso to choose the features and then refit those three by OLS.

Problem 2.3: The Regularization Path

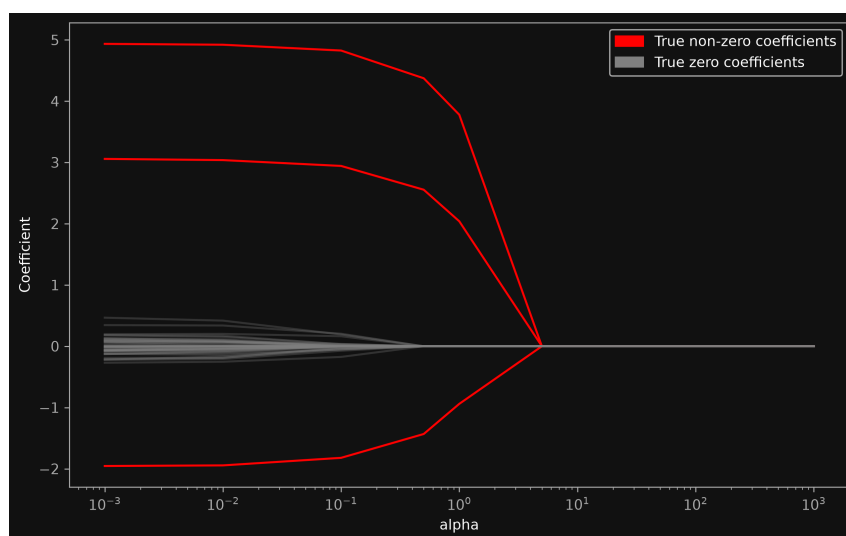
The sparsity of the model depends heavily on the strength of α .

- Create a list of alphas: [0.001, 0.01, 0.1, 0.5, 1.0, 5.0, 10, 100, 1000].
- Loop through these alphas, fitting a Lasso model for each.
- For each alpha, store the values of all coefficients.

Plot a graph with $\log(\alpha)$ on the x-axis and the coefficients on the y-axis (you can plot each coefficient as a separate line). Discuss how the coefficients change as alpha increases, and what the bias looks like (how far are the first 3 coefficients from their true values?).

Hint: It might be useful to plot the irrelevant coefficients in gray, and the relevant ones in different colors.

Answer:



α	X_1 (true 5)	X_2 (true -2)	X_3 (true 3)
0.001	4.9344	-1.9543	3.0572
0.010	4.9204	-1.9444	3.0375
0.100	4.8256	-1.8210	2.9424
0.500	4.3733	-1.4323	2.5543
1.000	3.7783	-0.9403	2.0392
5.000	0.0000	0.0000	0.0000

The noise coefficients are gone by $\alpha \approx 1$; the real ones survive to somewhere between 1 and 5. That gap is the window in which Lasso does feature selection.

The bias is a shift, not a scaling. Going from $\alpha = 0.001$ to $\alpha = 1$ moves X_1 by 1.156, X_2 by 1.014 and X_3 by 1.018 – each coefficient loses roughly α in absolute value, which is soft-thresholding.

That is why the smallest true coefficient, -2 , is the worst hit in relative terms: it has lost 53% of its size at $\alpha = 1$ while X_1 has lost 24%. It is also why every coefficient hits zero at once here, rather than fading out.

Problem 3: Time Series and Time-Varying Heteroskedasticity

Generate a dataset with $n = 1000$ observations. The data follows an AR(1) process:

$$y_t = 0.5y_{t-1} + \epsilon_t$$

The error term ϵ_t follows an ARCH(1) process plus a regime:

$$\epsilon_t = \sigma_t z_t, \quad z_t \sim \mathcal{N}(0, 1)$$

$$\sigma_t^2 = 1 + 0.5\epsilon_{t-1}^2 + 10 \cdot \mathbb{I}_{\text{HighRegime}}$$

Use the following code snippet to generate your data:

```

1 import numpy as np
2 import pandas as pd
3
4 np.random.seed(100)
5 n = 1000
6
7 indicator = np.zeros(n)
8 for i in range(0, n, 200):
9     indicator[i:i+100] = 1
10
11 epsilon = np.zeros(n)
12 sigma2 = np.zeros(n)
13 epsilon[0] = np.random.normal(0, 1)
14
15 for t in range(1, n):
16     sigma2[t] = 1 + 0.5 * (epsilon[t-1]**2) + 10 * indicator[t]
17     epsilon[t] = np.random.normal(0, np.sqrt(sigma2[t]))
18
19 y = np.zeros(n)
20 phi = 0.5
21 for t in range(1, n):
22     y[t] = phi * y[t-1] + epsilon[t]
```

Problem 3.1: The AR(1) Model

First, fit an AR(1) model to the generated series y_t .

$$y_t = \phi y_{t-1} + \epsilon_t$$

Extract the residuals ϵ_t from this model.

Create two plots:

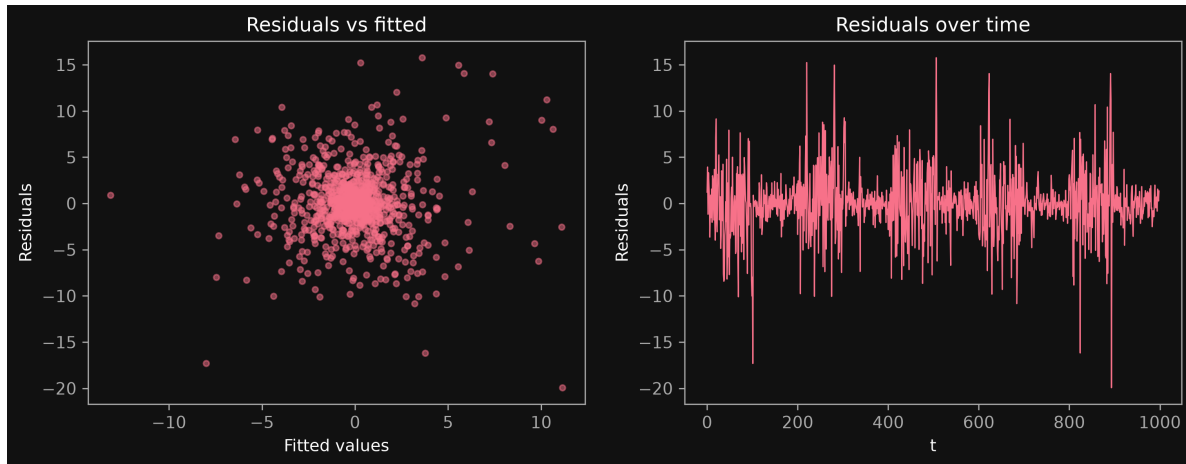
- Residuals vs. fitted values.

- Residuals over time.

What do you notice about the first graph vs. the second graph?

Answer:

$\hat{\phi} = 0.5182$ against a true 0.5, so the conditional mean is recovered.



Residuals versus fitted values shows nothing: an even band, no funnel, no curvature. That plot is blind here, because it sorts the residuals by fitted value and the variance does not depend on the fitted value.

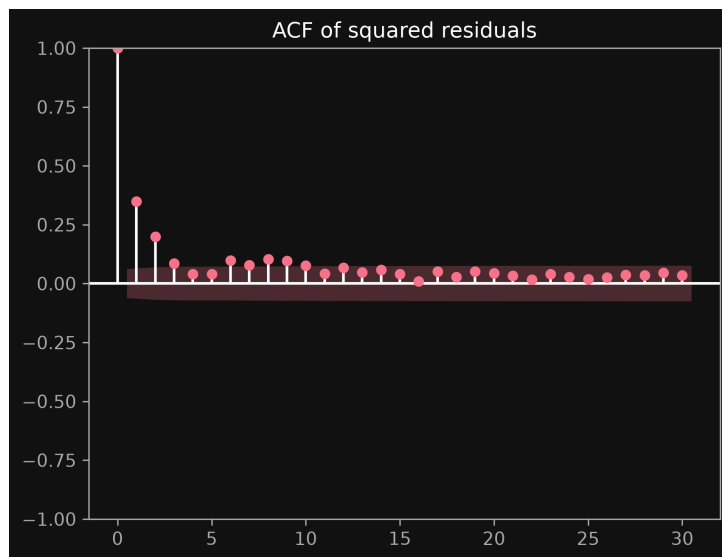
Residuals over time shows everything: alternating stretches of wild and quiet, on a period of about 200 steps. The variance depends on t , not on $\hat{y}$, so ordering the residuals by time is the only way to see it.

Problem 3.2: ACF

Plot the Autocorrelation Function (ACF) of the squared residuals $\hat{\epsilon}_t^2$.

What does the ACF plot suggest about the independence of the squared residuals?

Answer:



The squared residuals are strongly autocorrelated: the first several lags sit well outside the confidence band, and the decay is slow.

The residuals themselves are uncorrelated – the AR(1) took care of that – but their squares are not, so the residuals are not independent. A big move today predicts a big move tomorrow, in magnitude but not in sign. That is volatility clustering, and it is exactly what the ARCH term plus the regime put into the data.

Problem 3.3: Modeling Variance (ARCH + Regime)

Now, we will model the variance of these residuals explicitly. We hypothesize that the variance σ_t^2 depends on the previous squared residual (ARCH effect) and the regime indicator.

- Construct the squared residuals $\hat{\epsilon}_t^2$.
- Construct a lagged feature $\hat{\epsilon}_{t-1}^2$.
- Run a linear regression of $\hat{\epsilon}_t^2$ on:

$$\sigma_t^2 = \beta_0 + \beta_1 \hat{\epsilon}_{t-1}^2 + \beta_2 \mathbb{I}_{\text{HighRegime}}$$

- Report the coefficients, compare them to the true values ($\beta_0 = 1$, $\beta_1 = 0.5$, $\beta_2 = 10$), and discuss their statistical significance.

Answer:

	coef	std err	t	P> t	[0.025	0.975]
const	2.9868	1.233	2.422	0.016	0.567	5.406
resid2_lag1	0.2994	0.030	9.927	0.000	0.240	0.359
indicator	11.3243	1.795	6.307	0.000	7.801	14.848

All three are significant, and both the ARCH term and the regime dummy overwhelmingly so ($t = 9.93$ and $t = 6.31$). The regression finds the right structure.

The point estimates are only roughly right: 2.9868 against $\beta_0 = 1$, 0.2994 against $\beta_1 = 0.5$, 11.3243 against $\beta_2 = 10$. The ARCH coefficient is the one that is clearly off, and the intercept absorbs what it gives up. Two reasons: $\hat{\epsilon}_{t-1}^2$ is a noisy proxy for ϵ_{t-1}^2 , which attenuates $\hat{\beta}_1$ toward zero in the usual errors-in-variables way; and $\hat{\epsilon}_t^2$ is an extremely skewed dependent variable, so OLS on it is inefficient and its coefficients are unstable in a sample of 998.

Problem 3.4: Regime-Dependent Forecasting

Suppose we are at the end of the series ($T = 1000$) and we want to predict the range of movement for the next step ($T = 1001$). Assume the last observed residual was $\hat{\epsilon}_{1000} = 2$.

Calculate the predicted variance $\hat{\sigma}_{1001}^2$ and the resulting 95% Confidence Interval width ($\pm 1.96\hat{\sigma}_{1001}$) for the error term under two scenarios:

- **Scenario A (Low Regime):** Assume the indicator for $T = 1001$ is 0.
- **Scenario B (High Regime):** Assume the indicator for $T = 1001$ is 1.

(Use the coefficients you estimated in 3.3).

What do you observe about your confidence intervals in the two scenarios?

Answer:

With $\hat{\epsilon}_{1000} = 2$, so $\hat{\epsilon}_{1000}^2 = 4$:

$$\hat{\sigma}_{1001}^2 = 2.9868 + 0.2994 \times 4 + 11.3243 \times \mathbb{I}.$$

	$\hat{\sigma}_{1001}^2$	Lower Bound	Upper Bound
Low Regime	4.1846	-4.0094	4.0094
High Regime	15.5088	-7.7187	7.7187

The same model, the same last observation, and the interval is 1.93 times wider in the high regime. Nothing about the point forecast changed – only the regime flag.

A single unconditional interval would be wrong in both directions at once: too wide in the quiet regime and too narrow in the loud one, which is the worse of the two errors.

Problem 4: Weighted Least Squares

This problem focuses on verifying the findings of Class 3 regarding Weighted Least Squares (WLS).

Problem 4.1: Setup

Similar to before, define the true model as:

$$y = 2x$$

Where x is sampled from the uniform distribution $x \sim \text{Uniform}(1, 2)$.

Second, define the noise ϵ_i to be sampled from a normal distribution with mean 0, and variance σ_i^2 that depends on x_i as follows:

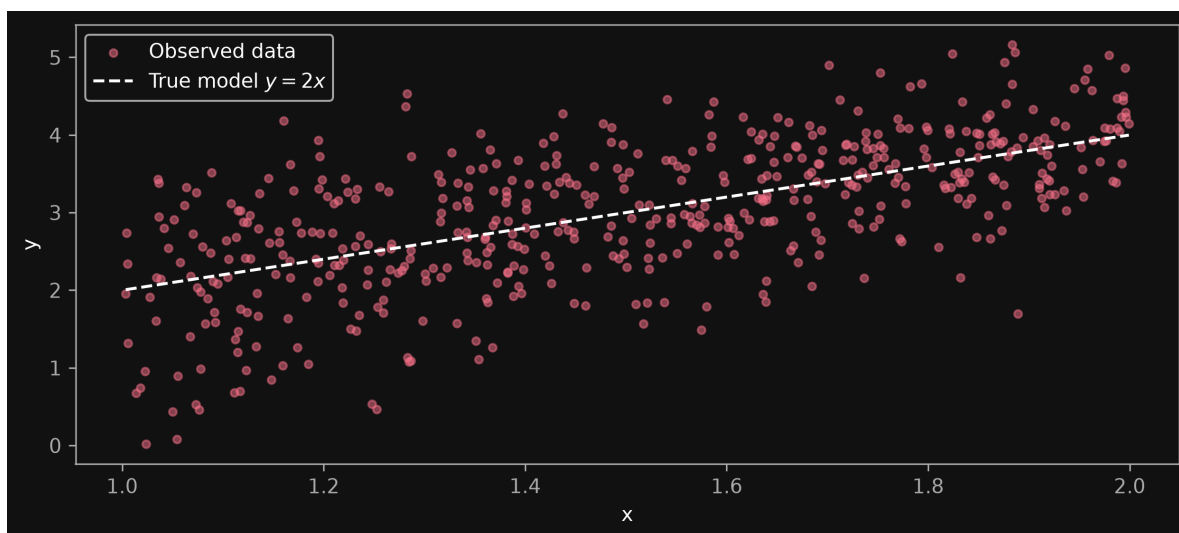
$$\sigma_i^2 = \frac{1}{x_i^2}$$

For $n = 500$, please plot the generated data points $(x_i + \epsilon_i, y_i)$.

Answer:

The true model has no intercept, so every fit below is through the origin, and the noise enters the response: $y_i = 2x_i + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, 1/x_i^2)$. That is what makes $w_i = x_i^2$ the correct GLS weights in 4.5, since $w_i = 1/\sigma_i^2$.

```
1 np.random.seed(500)
2 n = 500
3
4 x = np.random.uniform(1, 2, n)
5 sigma = 1 / x
6 epsilon = np.random.normal(0, sigma)
7 y = 2 * x + epsilon
```



The vertical spread is widest at $x = 1$ ($\sigma_i = 1$) and narrowest at $x = 2$ ($\sigma_i = 1/2$), which is the heteroskedasticity we built in.

Problem 4.2: Naïve OLS

Fit a standard OLS regression to the data generated above.

Report your estimate $\hat{\beta}_{OLS}$, and the confidence interval for the estimate.

Answer:

	coef	std err	t	P> t	[0.025	0.975]
x	1.9975	0.021	96.001	0.000	1.957	2.038

$\hat{\beta}_{\text{OLS}} = 1.9975$, with a 95% interval of $[1.9566, 2.0384]$. The point estimate is fine – OLS is still unbiased under heteroskedasticity. What is wrong is the standard error.

Problem 4.3: Interpretation

Briefly discuss whether naive OLS will be under or over confident in its estimate of $\hat{\beta}_{\text{OLS}}$, and why.

Answer:

Under-confident: the reported interval is wider than it needs to be.

For a fit through the origin, $\hat{\beta} - \beta = \sum_i x_i \epsilon_i / \sum_i x_i^2$, so the true variance is

$$\text{Var}(\hat{\beta}_{\text{OLS}}) = \frac{\sum_i x_i^2 \sigma_i^2}{(\sum_i x_i^2)^2} = \frac{\sum_i x_i^2 \cdot (1/x_i^2)}{(\sum_i x_i^2)^2} = \frac{n}{(\sum_i x_i^2)^2},$$

whereas OLS reports $\bar{\sigma}^2 / \sum_i x_i^2$, charging every point one common variance. The ratio of reported to true variance is

$$\frac{\bar{\sigma}^2 \sum_i x_i^2}{n} \rightarrow \mathbb{E}\left[\frac{1}{x^2}\right] \mathbb{E}[x^2] = \frac{1}{2} \cdot \frac{7}{3} = \frac{7}{6},$$

so the reported standard error is about $\sqrt{7/6} \approx 1.08$ times too large.

In a fit through the origin the large- x points carry the most weight, and those are exactly the points with the *smallest* noise, since $\sigma_i = 1/x_i$. OLS charges them the average noise level instead, so it overstates its own uncertainty. The heteroskedasticity-robust (HC3) standard errors from Class 2 confirm it:

	$\hat{\beta}$	std err	CI lower	CI upper
OLS (conventional)	1.99752	0.02081	1.95664	2.03840
OLS (HC3 robust)	1.99752	0.01962	1.95907	2.03596

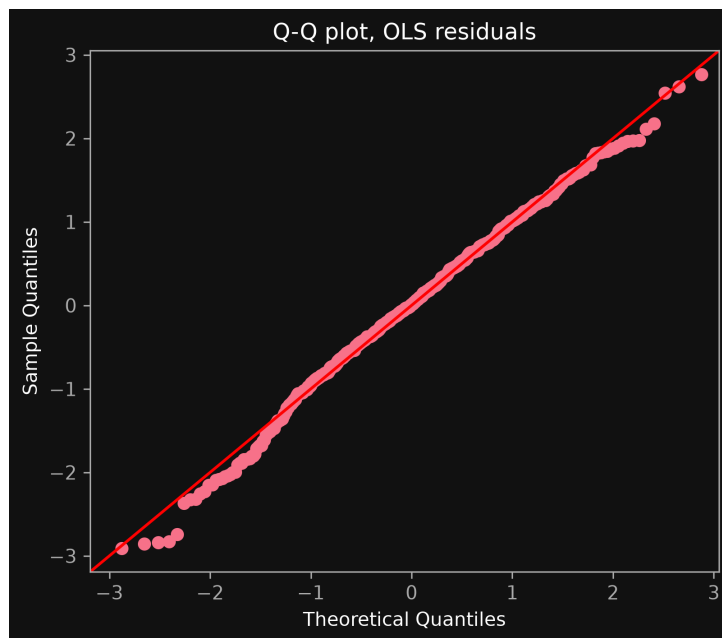
The realized ratio of the two standard errors is 1.0608, against the 1.0801 the algebra predicts in the limit.

Note that the direction is specific to this design. If the variance *grew* with x instead, the heavily weighted points would be the noisy ones and naive OLS would be over-confident.

Problem 4.4: Visualization

Plot a Q-Q plot of the residuals from the naive OLS regression. What do you observe?

Answer:



The points bend away from the 45° line in both tails. Each ϵ_i is normal on its own, but the residuals pool draws from 500 *different* variances – a scale mixture of normals, which is not itself normal. Shapiro-Wilk gives $p = 0.0482$, which rejects at the 5% level.

Problem 4.5: Weighted Least Squares

Fit a Weighted Least Squares regression to the data generated above, using weights $w_i = x_i^2$. Report your estimate $\hat{\beta}_{\text{WLS}}$, and the confidence interval for the estimate.

Answer:

	coef	std err	t	P> t	[0.025	0.975]
x	2.0024	0.018	108.781	0.000	1.966	2.039

$\hat{\beta}_{\text{WLS}} = 2.0024$, with a 95% interval of $[1.966, 2.039]$. The weights $w_i = x_i^2 = 1/\sigma_i^2$ are exactly the GLS weights, so by Gauss-Markov this is the efficient estimator: its standard error of 0.018 beats both the conventional and the robust OLS numbers from 4.3.

Problem 4.6: Visualization

Plot a Q-Q plot of the weighted residuals from the WLS regression. What do you observe compared to the Q-Q plot from the naive OLS regression?

Answer:



Naive OLS



WLS, weighted residuals

Much straighter, and the reason is exact: the weighted residual is $\sqrt{w_i} \epsilon_i = x_i \epsilon_i$, which has variance $x_i^2 \cdot (1/x_i^2) = 1$ for *every* i . Weighting turns the scale mixture back into a single standard normal.

	std dev	Shapiro p	$\text{corr}(\hat{\epsilon} , x)$
OLS residuals	0.7128	0.0482	-0.2224
Weighted (WLS) residuals	1.0293	0.3125	0.0292

The weighted residuals have a sample standard deviation of 1.0293 against a theoretical 1, Shapiro-Wilk no longer rejects, and the correlation between $|\hat{\epsilon}_i|$ and x_i has collapsed from -0.2224 to 0.0292.

Note that `model.wls.resid` returns the *raw* residuals $y_i - x_i \hat{\beta}$, which are still heteroskedastic. The weighted ones are `model.wls.wresid`.